Aurora Tickets

Raúl Mateo Azparren

Índice

Índice 1
1 Business Understanding 2
Objetivos de negocio 2
KPIs 2
Decisiones 2
2 Data Understanding 3
Fuente de datos 3
Clickstream 3
Datos de negocio (CSV) 3
Estructura Logs 3
Volumen y temporalidad 3
3 Data Preparation 4
Limpieza de datos 4
Transformaciones 4
Data Lake S3 4
Formato y particionado 4
4 Modeling 4
Arquitectura 4
Job 1 5
Job 2 6
Productos analíticos 8
Visualización 9
5 Evaluación 10
Resultados 10
s3 10
RDS 11
Limitaciones 11
Deployment 11
Arquitectura AWS 11
Reproducibilidad 13
Estimación de costes 14

1 Business Understanding

Aurora Tickets es una plataforma de venta de entradas que present que esta experimentando un crecimiento descontrolado.Estos son los principales problemas:

- Alto tráfico web pero baja conversión
- Problemas de rendimiento (errores y latencia)
- Posible tráfico fraudulento o bots
- Falta de observabilidad y analítica avanzada

Objetivos de negocio

Mejorar la tasa de conversión
Identificar eventos con alto interés pero bajo ingreso
Detectar anomalías o comportamiento sospechoso
Facilitar la toma de decisiones basada en datos

KPIs

Para medir el éxito del proyecto, se han definido los siguientes indicadores que se visualizarán en el Dashboard de CloudWatch y en el análisis final:

- Conversion rate (sesiones → compra)
- Ingresos por evento
- Ratio interés vs compra
- Número de anomalías detectadas

Decisiones

Para lograr estos objetivos, se desplegará una arquitectura en AWS compuesta por:

- Un clúster de Apache Spark(1 Master, 3 Workers) para el procesamiento
- distribuido.
- Un Data Lake en S3 organizado en capas (Raw, Curated, Analytics).
- CloudWatch Logs para la ingesta de eventos en tiempo real.
- RDS (MySQL) para el almacenamiento de los resultados finales.

2 Data Understanding

Fuente de datos

Los datos son generados por un simulador de tráfico que emula el comportamiento de usuarios reales y bots en la web de "Aurora Tickets".

Clickstream

- Eventos de navegación (client-side)
- Logs del backend (server-side)

Datos de negocio (CSV)

- events.csv: catálogo de eventos
- campaigns.csv: campañas marketing (UTM)
- transactions.csv: compras

Estructura Logs

Cada evento de navegación sigue un formato JSON (o semi-estructurado en log) con los siguientes campos:

- timestamp: Fecha y hora del evento.
- user_id / session_id: Identificadores únicos para rastrear el embudo de ventas.
- method: Acción realizada (GET/POST).
- endpoint: La página visitada (/home, /tickets, /checkout,
- /pago_confirmado).
- status_code: Respuesta del servidor (200 OK, 404 Not Found, 500 Error).
- ticket_type: Categoría de la entrada si el evento es una selección (A, B o C).
- ip_address: Vital para identificar patrones de bots (muchas peticiones desde una
- sola IP).
- user_agent: Navegador/dispositivo (ayuda a distinguir humanos de scripts).

Volumen y temporalidad

- ≥ 200.000 eventos
- Simulación de 7 días
- Mezcla de generación masiva + tráfico real

3 Data Preparation

Limpieza de datos

- Eliminación o imputación de nulos
- Corrección de tipos (fechas, numéricos)
- Eliminación de registros inconsistentes
- Filtrado de outliers en ingresos
- Resolución de claves huérfanas

Transformaciones

- Normalización de esquemas
- Enriquecimiento de clickstream con datos de negocio (joins)
- Generación de métricas intermedias

Data Lake S3

Estructura:

- raw/ → datos originales
- curated/ → datos limpios y estructurados
- analytics/ → datos agregados finales

Formato y particionado

- Formato: Parquet
- Particionado por: dt=YYYY-MM-DD

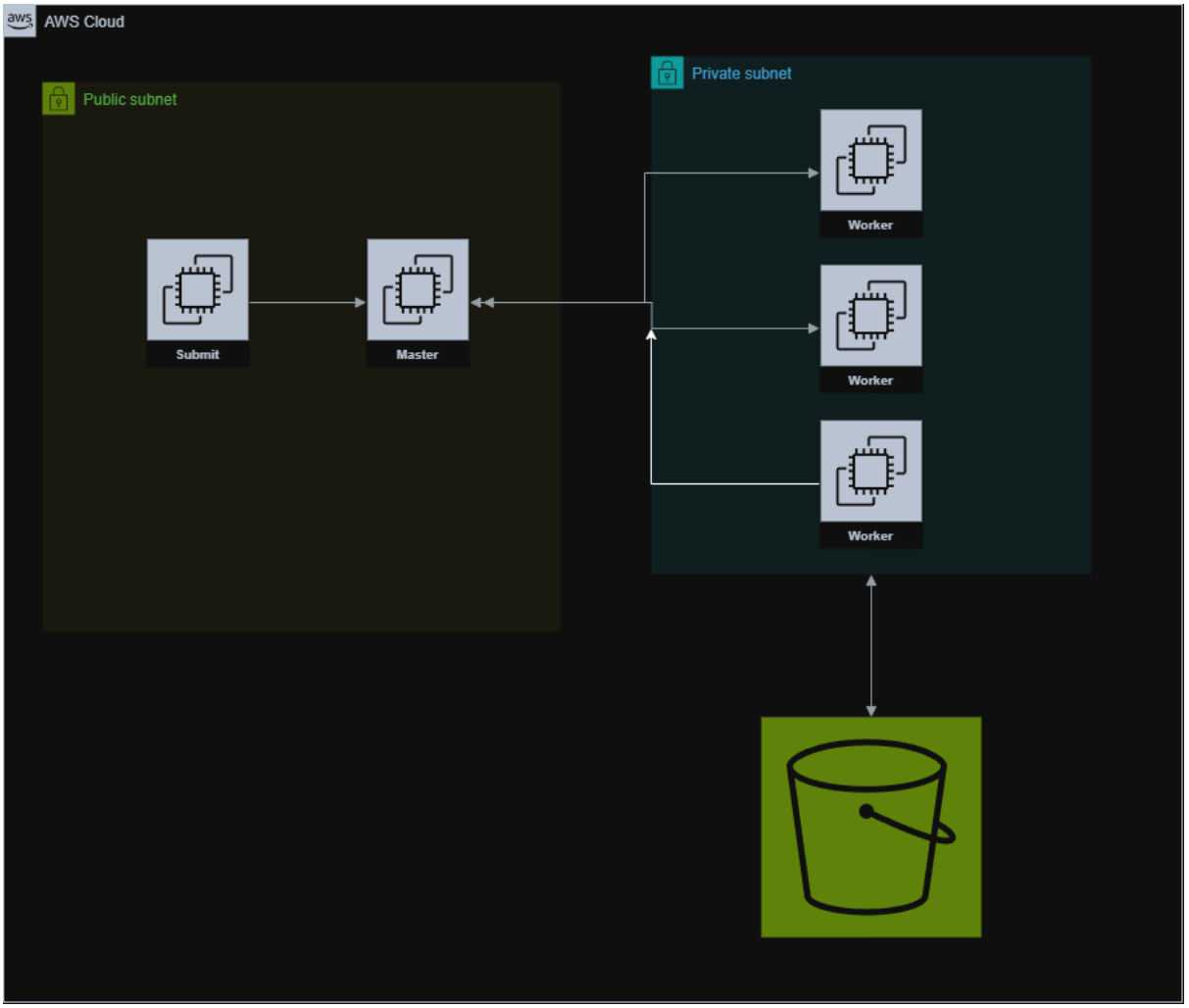
4 Modeling

Arquitectura

- Cluster Spark Standalone (1 master + 3 workers)
- Nodo submit para ejecución de jobs
- Ingesta desde CloudWatch → S3

Job 1

- Entrada: raw/
- Procesos:
    - Limpieza
    - Normalización
    - Join entre datasets
- Salida: curated/ (Parquet particionado)

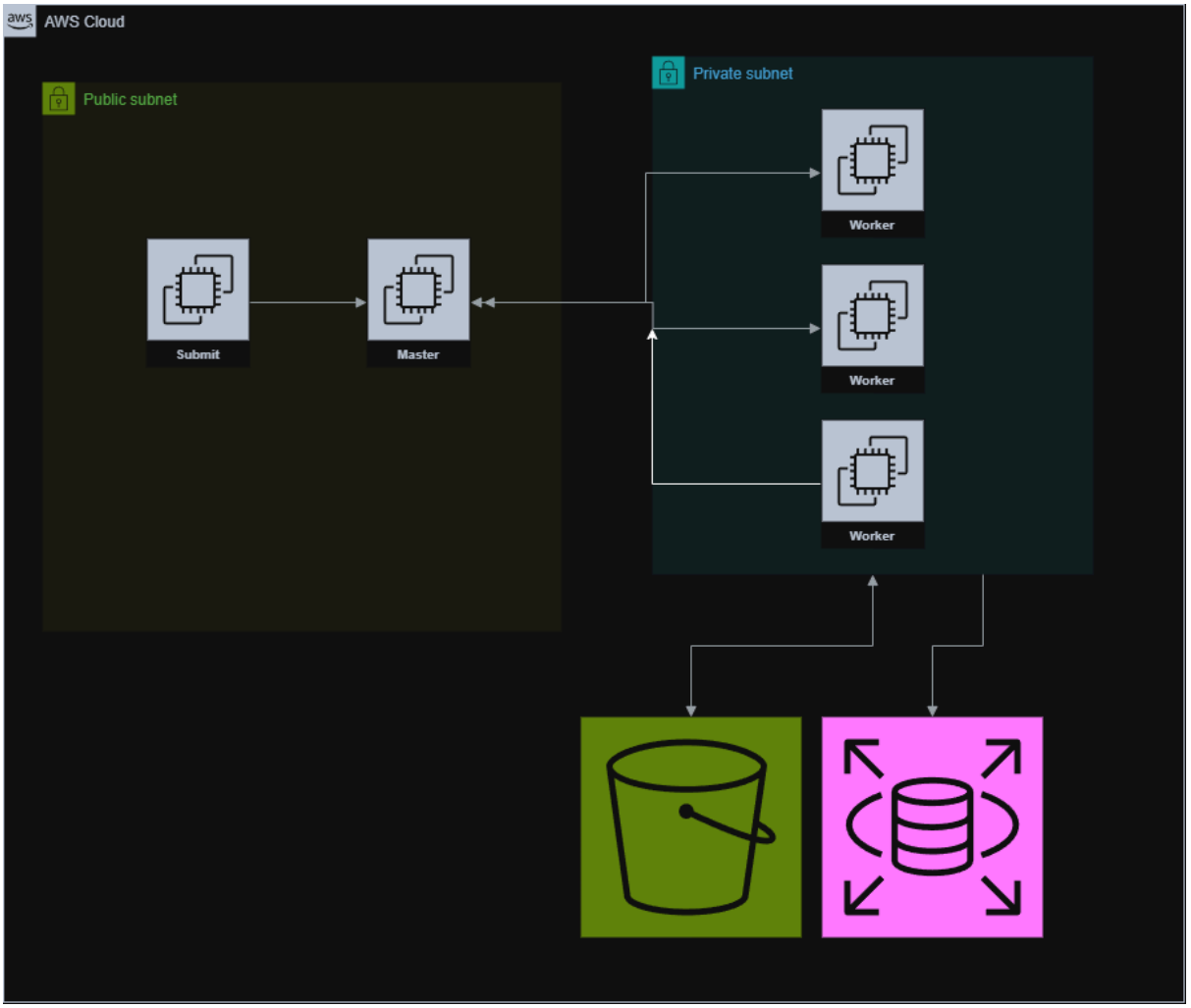


job1.py > ...

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.window import Window

# ========================
# 1. SPARK SESSION
# ========================
spark = SparkSession.builder \
    .appName("events_etl") \
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .getOrCreate()

base_path = "s3a://aurora-10/raw/"

# ========================
# 2. READ
# ========================
events = spark.read.csv(base_path + "events.csv", header=True)
campaigns = spark.read.csv(base_path + "campaigns.csv", header=True)
transactions = spark.read.csv(base_path + "transactions.csv", header=True)
clickstream = spark.read.json(base_path + "aurora_clickstream.jsonl")

# ========================
# 3. CAST TIPOS
# ========================
events = events \
    .withColumn("event_id", col("event_id").cast("string")) \
    .withColumn("event_date", to_date("event_date")) \
    .withColumn("base_price", col("base_price").cast("double")) \
    .withColumn("capacity", col("capacity").cast("int")) \
    .withColumn("is_active", col("is_active").cast("boolean"))

```

Job 2

- Entrada: curated/
- Procesos:
  - Agregaciones
  - Cálculo de métricas
- Salidas:
  - analytics/ en S3
  - Tablas finales en RDS MySQ


AWS Cloud
Public subnet
Private subnet
Submit
Master
Worker
Worker
Worker

job2.py > ...

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("events_etl") \
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .getOrCreate()
df = spark.read.parquet("s3a://aurora-10/curated/")

# Consulta 1
funnel = df.groupBy("dt", "session_id").agg(
    max(when(col("event_type") == "page_view", 1).otherwise(0)).alias("view"),
    max(when(col("event_type") == "event_list", 1).otherwise(0)).alias("list"),
    max(when(col("event_type") == "event_detail", 1).otherwise(0)).alias("detail"),
    max(when(col("event_type") == "begin_checkout", 1).otherwise(0)).alias("checkout"),
    max(when(col("transaction_id").isNotNull(), 1).otherwise(0)).alias("purchase")
)

funnel_daily = funnel.groupBy("dt").agg(
    countDistinct("session_id").alias("sessions_total"),
    sum("list").alias("sessions_event_list"),
    sum("detail").alias("sessions_event_detail"),
    sum("checkout").alias("sessions_begin_checkout"),
    sum("purchase").alias("sessions_purchase")
).withColumn(
    "conversion_rate",
    col("sessions_purchase") / col("sessions_total")
)

funnel_daily.write.mode("overwrite").parquet("s3a://aurora-10/analytics/funnel/")
```

Productos analíticos

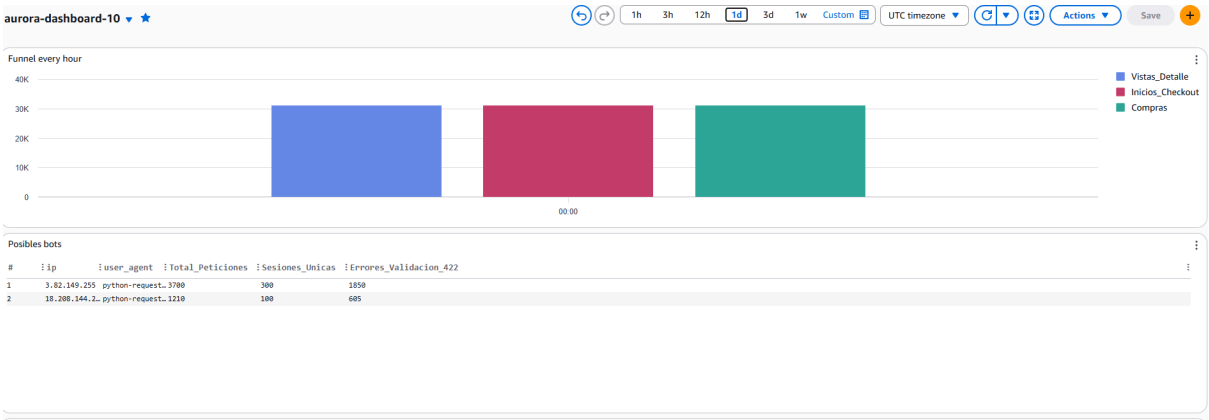
- Funnel
  - Métricas por día:
    - sesiones totales
    - paso por etapas (list → detail → checkout → purchase)
    - tasa de conversión
- Ranking de eventos
  - Métricas:
    - visitas a detalle
    - compras
    - ingresos
    - ratio interés/compra
- Detección de anomalías
  - Basado en:

        - número de requests por IP
        - errores
        - patrones inusuales
    - Regla:
        - IP con requests > umbral → anomalía

Visualización

CloudWatch Dashboard con:

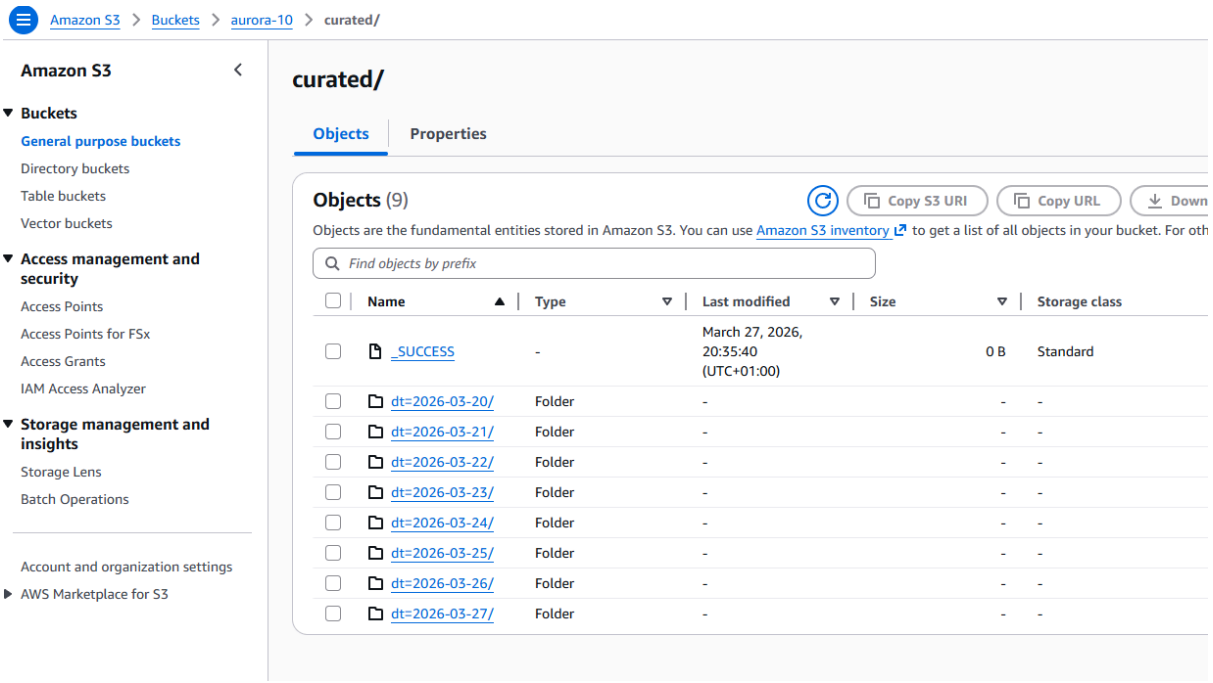
- Funnel
- Top eventos
- Errores/latencia
- Anomalías

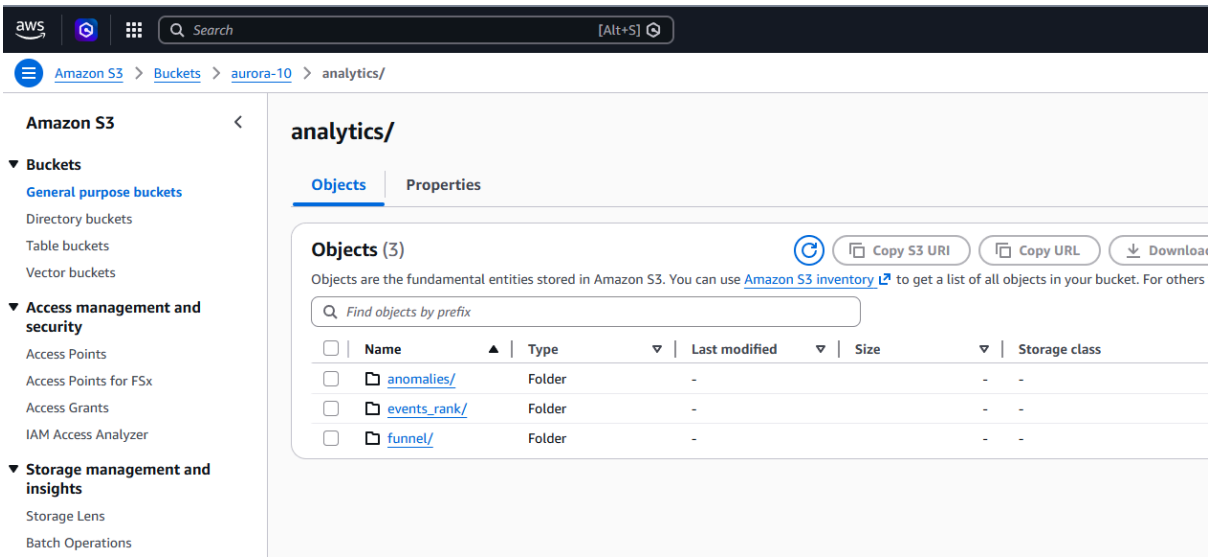


5 Evaluación

Resultados

s3





RDS

```
Query OK, 0 rows affected (0.020 sec)

MySQL [aurora]> show tables;
+----------------------+
| Tables_in_aurora     |
+----------------------+
| metrics_anomalies    |
| metrics_event_rank   |
| metrics_funnel_daily |
+----------------------+
3 rows in set (0.001 sec)

MySQL [aurora]>
```

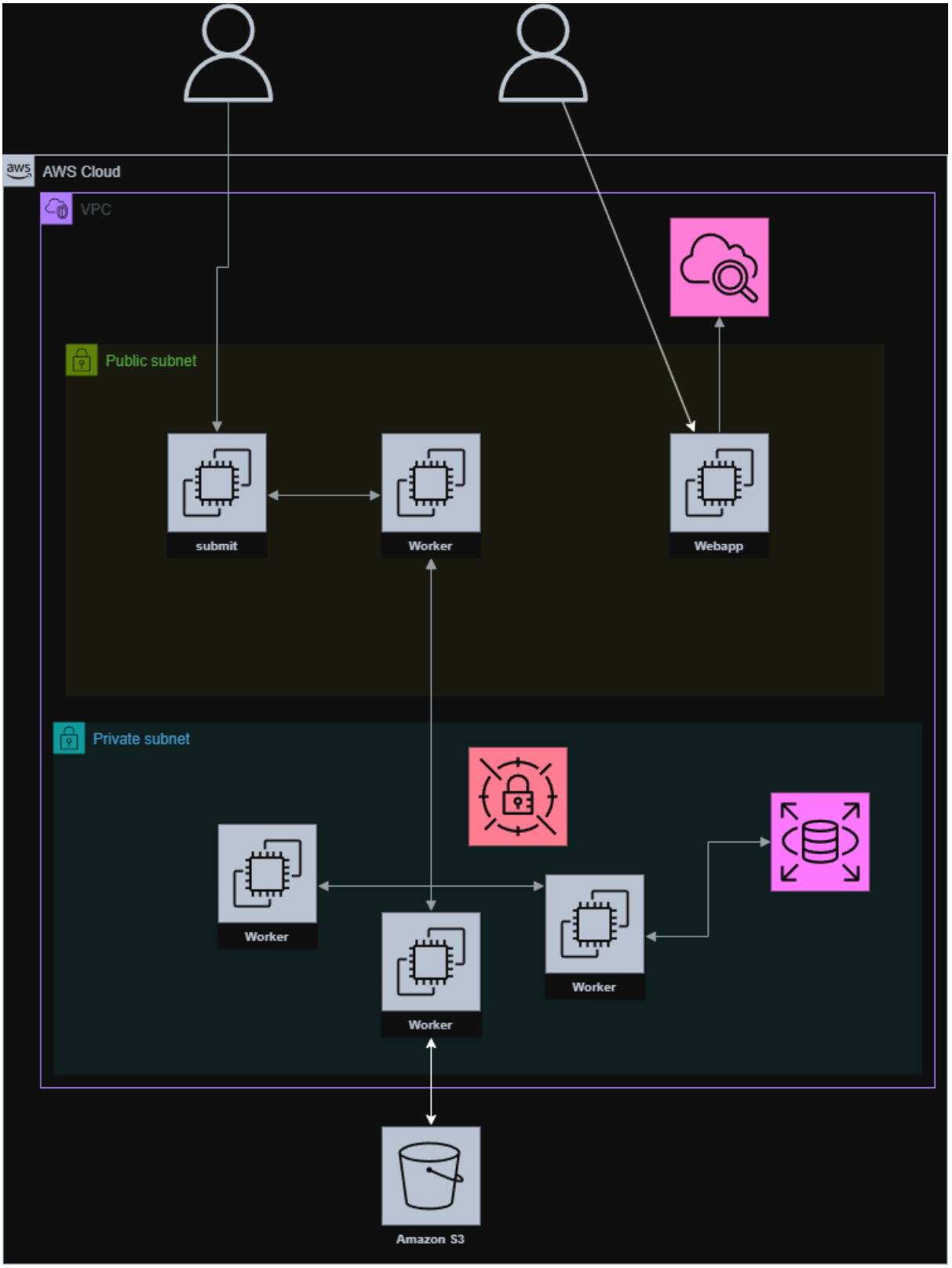
Limitaciones

- Datos simulados (no reales)
- Reglas de anomalía simples
- Pérdida de información en limpieza

Deployment

Arquitectura AWS

- EC2 (6 instancias):
  - 1 Master
  - 3 Workers
  - 1 Submit
- 1 Web + CloudWatch Agent
- S3 → Data Lake
- CloudWatch → logs + dashboard
- RDS MySQL → métricas finales


AWS Cloud
VPC
Public subnet
submit
Worker
Webapp
Private subnet
Worker
Worker
Worker
Amazon S3

Reproducibilidad

Para reproducir los datos tendríamos que irnos a la ec2 de la aplicacion web y desde la carpeta SBD/Tema 4 ejecutar los scripts para generar los datos de la empresa, simulacion de clicks y tráfico

```
python3 generators/generate_business_data.py \
--student-id 10 \
--seed 123 \
--days 7 \
--n-events 120 \
--n-campaigns 12 \
--n-transactions 20000 \
--error-rate 0.05 \
--orphan-rate 0.02 \
--out-dir ./output_business \
--frontend-data-dir ./webapp/frontend/data
```

```
python3 simulators/replay_clickstream.py \
--student-id 10 \
--days 7 \
--n-events 200000 \
--events-json ./webapp/frontend/data/events.json \
--campaigns-csv ./output_business/campaigns.csv \
--out /var/log/aurora/aurora_clickstream.jsonl \
--append
```

```
python3 simulators/traffic_driver.py \
--base-url http://<PUBLIC_IP> \
--student-id 10 \
--events-json ./webapp/frontend/data/events.json \
--campaigns-csv ./output_business/campaigns.csv \
--sessions 300 \
--max-actions 8 \
--sleep-ms 80
```

Ahora podremos ejecutar los jobs de spark en el nodo submit.

Estimación de costes

aws

Contact your AWS representative: Contact Sales

Export Date: 03/28/2026 **Language:** English

Estimate url

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	179.18 USD	2,150.16 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon Simple Storage Service (S3)	-	US East (N. Virginia)	0.00 USD	4.66 USD

Status -

Description: -

Config summary S3 Standard storage (200 GB per month), PUT, COPY, POST, LIST requests to S3 Standard (1000), GET, SELECT, and all other requests from S3 Standard (200), Data returned by S3 Select (20 GB per month), Data scanned by S3 Select (20 GB per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	-	US East (N. Virginia)	0.00 USD	7.52 USD

Status -

Description: -

Config summary Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 1), Advance EC2 instance (t3.small), Pricing strategy (Compute Savings Plans 3yr No Upfront), Enable monitoring (disabled), DT Inbound: Not selected (0 TB per month), DT Outbound: Not selected (0 TB per month), DT Intra-Region: (0 TB per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	-	US East (N. Virginia)	0.00 USD	101.18 USD

Status -

Description: -

Config summary Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 6), Advance EC2 instance (t2.medium), Pricing strategy (Compute Savings Plans 3yr No Upfront), Enable monitoring (disabled), DT Inbound: Not selected (0 TB per month), DT Outbound: Not selected (0 TB per month), DT Intra-Region: (0 TB per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon RDS for MySQL	-	US East (N. Virginia)	0.00 USD	54.13 USD

Status -

Description: -

Config summary Storage amount (20 GB), Storage for each RDS instance (General Purpose SSD (gp2)), Nodes (1), Instance type (db.t4g.micro), Utilization (On-Demand only) (100 %Utilized/Month), Deployment option (Single-AZ), Pricing strategy (OnDemand)

Name	Group	Region	Upfront cost	Monthly cost
AWS Secrets Manager	-	US East (N. Virginia)	0.00 USD	0.40 USD

Status -

Description: -

Config summary Number of secrets (1), Average duration of each secret (30 days), Number of API calls (50 per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon CloudWatch	-	US East (N. Virginia)	0.00 USD	11.29 USD

Status